

Verification Continuum™

Siloti® Correlation
User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	5
Customer Support	5
Synopsys Statement on Inclusivity and Diversity	6

1. Introduction	7
Overview	7
Verdi Power Analysis Acceleration Overview	7

2. Siloti Correlation Overview	8
Siloti Correlation Mapping RTL and Gate-Level Design	8
Leverage SVF Output from Design Compiler	9
Generating CRDB	12
CRDB Utility	12
Exclude Module for Macro Cell	16
Filtering for Gate Only Register	16
CRDB Environment Variable	17
CR_SKIP_HIER_FILE	17
Third-Party Tool with Siloti Correlation	18
Cross-Abstraction Debug	19

3. Siloti Correlation Practice	20
Before You Begin	20
Debug Gate-Level Results on the RTL Design	21
Input Requirements	21
Extract Correlation Database	21
Create Mapping File (Optional)	22
Expand Data and Debug on the RTL Design	23
Verdi Power Analysis Acceleration Flow	25

4. Name Mapping Rules	28
------------------------------------	-----------

Contents

Regular Expression Examples 29

Pre-defined Rules Examples30

Applying Rules 31

Preface

This book is designed to help you quickly become proficient with Siloti Correlation Engine. The manual should be read from beginning to end, although you may skip any sections with which you are already familiar. The setup and options for Siloti Correlation are described in this document. By using this document, you can understand how to setup and run design correlation with the Siloti Correlation engine.

Audience

The audience for this manual includes engineers who look for the details of the commands and options available of Siloti Correlation.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to

identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

In this chapter:

- [Overview](#)
- [Verdi Power Analysis Acceleration Overview](#)

Overview

Siloti Correlation is a technology that allows you to map signals between the RTL and gate-level design automatically, performing debug automation across abstraction levels.

Siloti correlation uses readily available RTL simulation results in the early stage of the flow rather than gate-level simulation results and shorten the runtime while maintaining the accuracy.

Verdi Power Analysis Acceleration Overview

Verdi Power Analysis Acceleration uses Siloti Correlation and Siloti Replay Simulation technologies to help user run the gate level simulation and dump necessary signals waveform for power analysis tools to calculate the power consumption.

To obtain the most accurate power analysis results, users usually use the later design stage of simulation result as the switching activity data for the power analysis. However, due to the runtime of gate level simulation that takes long time, users face the challenges to obtain the gate level simulation for the power analysis. With Siloti Correlation technology, users can use the RTL simulation result to correlate to the gate level design. Siloti Replay Simulation takes the input from the RTL simulation and run the gate level simulation to obtain the gate level simulation result for the power analysis.

2

Siloti Correlation Overview

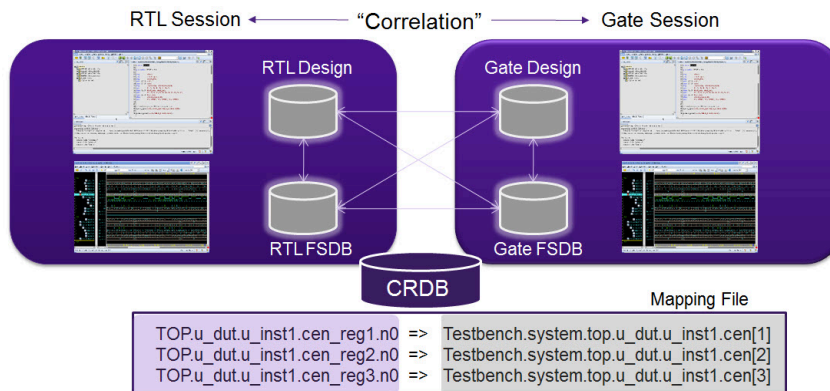
In this chapter:

- [Siloti Correlation Mapping RTL and Gate-Level Design](#)
- [Generating CRDB](#)
- [Third-Party Tool with Siloti Correlation](#)
- [Cross-Abstraction Debug](#)

Siloti Correlation Mapping RTL and Gate-Level Design

The Verdi Automated Debug Platform provides the Siloti correlation methodology of cross-abstraction for correlation and debug. Siloti correlation maps signals between the RTL and gate-level design automatically, generate the RTL and gate-level design mapping file, and debug across abstraction levels, such as value annotations, and tracing seamlessly.

Figure 1 *Siloti Correlation Maps Signals between RTL and Gate-Level Design*



Nowadays, designs frequently come across challenges including the requirement of post-layout simulation for power estimation, time-consuming re-purposing RTL functional test on gate-level netlist, and re-simulations starting from time zero that waste significant simulation time.

By mapping signals between the RTL and gate-level design, Siloti correlation can use readily available RTL simulation results in the early stage of the flow rather than gate-level simulation results and shorten the runtime while maintaining the accuracy.

The Siloti methodologies include the following:

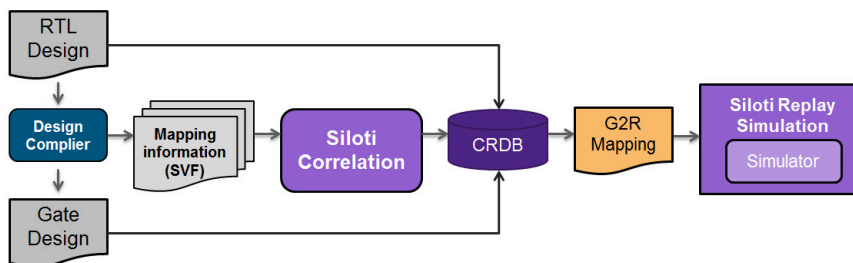
- Use the RTL and netlist designs.
- Leverage SVF output from the design compiler.
- Generate mapping file.

Leverage SVF Output from Design Compiler

Siloti correlation leverages the output of the SVF engine from the design compiler by reading the synthesis mapping information to increase mapping rate. The SVF output contains the information about the detailed synthesis naming rules and optimization. Siloti correlation performs for the name change, hierarchy change, optimization, and CTS that are recorded in the SVF file automatically.

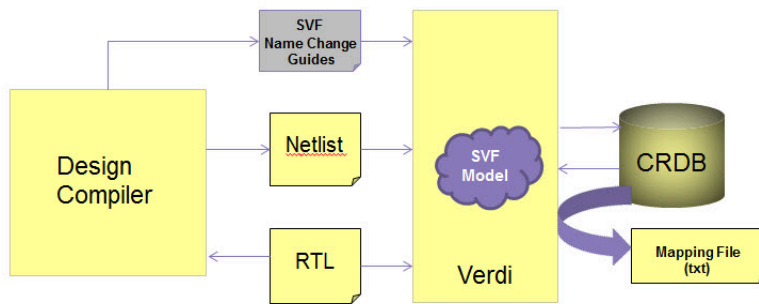
By leveraging the SVF output, Siloti correlation provides the flexibility as it supports reading synthesis mapping information to increase the mapping rate and accepts the mapping file that you provide. Also, Siloti correlation maps signals between the RTL and gate-level design, saving the resource for the manual maintenance of mapping rule. In addition, the high mapping rate between the RTL and gate-level design provide accuracy for power analysis.

Figure 2 Siloti Correlation Leverages SVF Output From Design Compiler



Siloti correlation features the abstraction correlation module that automatically maps design descriptions and verification results in one abstraction level to its counterpart in another abstraction level, such as the register net mapping from the gate-level to RTL, or the RTL to gate-level.

Figure 3 SVF Flow in Verdi



The following are the complete set of SVF guide commands:

Figure 4 Supported SVF Guide Commands

guide	guide_reg_removal	guide_fsm_reencoding	guide_merge
guide_netlist_table	guide_datapath	guide_scan_input	guide_ungroup
guide_architecture_db	guide_reg_split	guide_group	guide_mim_retiming
guide_pins_eqop	guide_dont_verify_scan	guide_scan_output	guide_uniquify
guide_architecture_netlist	guide_rename_design	guide_group_function	guide_multibit
guide_port_constant	guide_dsp_implementation	guide_sequential_cg_fanin	guide_ununiquify
guide_arithmetic_fracture	guide_replace	guide_implementation	guide_multiplier
guide_port_punch	guide_dsp_netlist	guide_sequential_cg_fanout	guide_upf_copy
guide_block_merging	guide_retiming	guide_info	
guide_reg_constant	guide_dsp_pack	guide_set_rounding	
guide_boundary	guide_retiming_decompose	guide_instance_map	
guide_reg_duplication	guide_eco_change	guide_share	
guide_boundary_netlist	guide_retiming_dw_pipeline	guide_inv_push	
guide_reg_encoding	guide_eco_map	guide_timebegin	
guide_change_names	guide_retiming_finished	guide_mark	
guide_reg_eqop	guide_environment	guide_timeend	
guide_checkpoint	guide_retiming_multibit	guide_mc	
guide_reg_merging	guide_exec	guide_transformation	
guide_constraints	guide_rewire	guide_merge	

The following are the supported SVF guide commands that impact the Verdi flow:

Figure 5 *Supported SVF Guide Commands Impacting Verdi Flow*

Guide Tcl Commands	Brief Description	Impact Verdi Flow
guide_rename_design	Renames a design	Yes
guide_change_names	Changes the name of design objects	Yes
guide_inv_push	Indicate the named register is phase inverted (inversion pushed)	Yes
guide_uniquify	Creates unique instances for design objects	Yes
guide_ununiquify	Folds uniquified instances back to their original designs.	Yes
guide_instance_map	Specifies a new design name for specially named designs	Yes
guide_ungroup	Removes hierarchy from design objects	Yes
guide_group	Creates hierarchy around design objects	Yes
guide_reg_constant	Asserts that a register can never reach any known value different from a specified constant value	Yes
guide_reg_duplication	Identifies duplicated registers.	Yes
guide_reg_merging	Identifies merged registers.	Yes
guide_multibit	Support name-matching of multibit banked registers.	Yes

The Siloti correlation mapping engine is heuristics-based, using built-in heuristic rules to correlate the RTL and gate-level design. However, the desired mapping rate may not be achieved in some cases due to its heuristic nature of the Siloti correlation mapping engine. Also, it may significant resource to achieve the acceptable mapping rate.

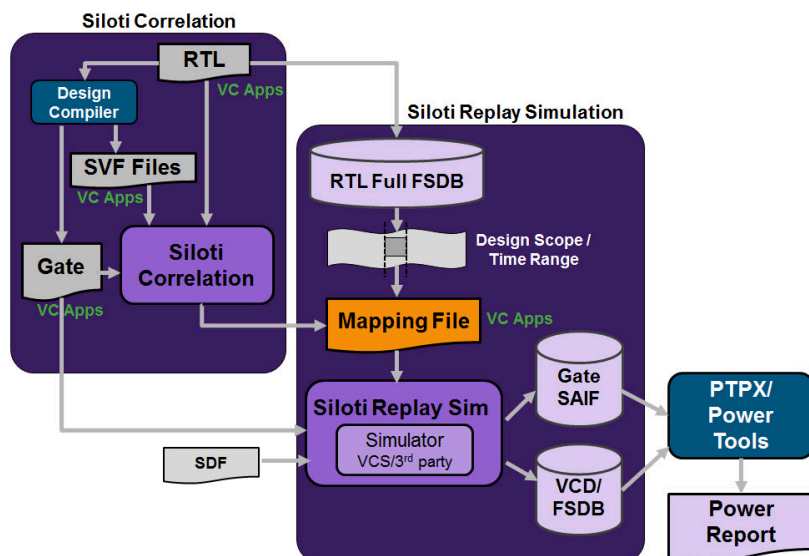
Fortunately, reading the design compiler output (SVF) files directly makes the overall flow more efficient for the user. The `crdb` utilities take the SVF file as the input along with the RTL/GATE designs, and write out the CRDB/mapping file. The Siloti correlation focuses on the register mapping only in short term. The following command line can be used to read the design compiler output (SVF) files:

```
%> crdb -RTL -f rtl.f -GATE -f gate.f -impSVF rtlopt.svf dcopt.svf -crdb
use_svf_map.crdb
```

The Siloti correlation can also take user-defined mapping files as it dumps text-based mapping files for the replay simulation to use. The users can always edit the mapping flow in the mapping file if the design modifications are not recorded in the SVF file.

VC Apps can also be used to customize the mapping flow in the Siloti correlation to fit in the original flow or characterized needs. VC Apps extracts the necessary data from the database and transforms the data to fit tool requirements. This helps you to automate the customization.

Figure 6 VC Apps Customizing Mapping Flow Help Fit in Original Flow



Generating CRDB

The CRDB provides the RTL to gate-level mapping data. It is a file that stores the design and the mapping information. It is read by different applications that provide debug and simulation capabilities. These applications include:

- Siloti application that provides the gate-level testbench instrumentation for what-if simulation using the RTL vectors.
- Drag and drop capability in Verdi between an application that displays the gate-level information and an application that displays RTL information. Vice versa.
- Annotation on Verdi applications to display mapped/correlated signals by showing special symbols.

The following utilities and environment variables can be used to generate and configure the CRDB file:

CRDB Utility

`crdb` is a batch mode executable for correlation database (CRDB) extraction. The results are saved to the `crdbLog` directory. Correlation extraction is performed for the specified working scope (using the `-bas` option). If the `-bas` option is not specified, `crdb` uses the working scope in the `novas.rc` resource file (if it exists). Otherwise, the top module of the design (Verilog only) is used.

Note:

The `crdb` utility requires a Siloti-SimVE license.

Note:

Set the `NOVAS_BATCH_KILL` environment variable to 1 when using the `crdb` utility in LSF environments.

Use Model:

```
crdb [CRDB Options] [General Options] [nTrace Options] [Simulator
Options] [Environment Options] [Behavior Analysis Options] [Power Manager
Options]
```

This table summarizes the CRDB options.

Options	Explanation
<code>-GATE</code> <code>"HDL_Elaboration_Options"</code>	Within the double quotes behind the <code>-GATE</code> option, specify the Behavior Analysis options needed to load the gate-level design. When this option is used with the <code>-RTL</code> option the complete CRDB is produced. Details are logged to the <code><logdir>/GATELog</code> directory. The default for <code><logdir></code> is <code>crdbLog</code> .
<code>-RTL</code> <code>"HDL_Elaboration_Options"</code>	Within the double quotes behind <code>-RTL</code> option, specify the Behavior Analysis options needed to load the RTL design. When this option is used with the <code>-GATE</code> option the complete CRDB will be produced. Details are logged to the <code><logdir>/RTLLog</code> directory. The default for <code><logdir></code> is <code>crdbLog</code> .
<code>-allreg</code>	When specified, all registers are mapped from the gate to the RTL for exporting the mapping file. This option must be used with the <code>-expMap</code> option and cannot be used with the <code>-fsdb</code> or the <code>-txt</code> options.
<code>-append</code>	Append data into the CRDB database. This can be used in an incremental build flow for large designs.
<code>-crdb filename</code>	Specify the working CRDB library name.
<code>-excludeModule filename</code>	All signals except port signals in excluded modules are excluded from the mapping file. The port signals in the excluded modules are listed in the mapping file. This option must be used with the <code>-expMap</code> option.
<code>-expGate</code>	When specified, the correlation analyzer will export gate objects from the mapping file. This option is optional and must be used with the <code>-expMap</code> option.

<code>-expGateUncor</code>	When specified, the correlation analyzer will export uncorrelated gate objects. This option is optional and must be used with the <code>-expMap</code> option.
<code>-expMap filename</code>	Export the mapping results to the specified file. This option must be used with only one of the <code>-fsdb</code> , <code>-txt</code> , or <code>-allreg</code> options.
<code>-expRtl</code>	When specified, the correlation analyzer will export RTL objects from the mapping file. This option is optional and must be used with the <code>-expMap</code> option.
<code>-filterModule filename</code>	When specified, all signals in filtered modules will be excluded from the uncorrelated signal list. A new mapping rate based on the filtered signals are reported. This option must be used with the <code>-expMap</code> option.
<code>-filterInstance filename</code>	When specified, all signals in filtered cell instances are excluded from the uncorrelated signal list. A new mapping rate based on the filtered signals is reported. Signal names (ports and nets) can also be specified directly. This option must be used with the <code>-expMap</code> option.
<code>-fsdb filename</code>	When specified, all signals in the specified FSDB file will be mapped from the gate to the RTL for exporting the mapping file. This option must be used with the <code>-expMap</code> option and cannot be used with <code>-txt</code> or <code>-allreg</code> .
<code>-h -help</code>	Print out the help message.
<code>-impMap filename</code>	Specify the user-defined mapping file. When specified, the correlation information from the mapping file is applied to the CRDB and the internal correlation algorithms are skipped.
<code>-impSVF filename1 [filename2...filenameN]</code>	Import the specified files of the synthesis tool guidance in the SVF format and the register correlation information from the specified SVF files are applied to the CRDB. Siloti Correlation naming rules apply to the remaining unmapped signals. Multiple files must be enclosed with square brackets.
<code>-novasLibPaths libpath1 [libpath2 ... libpathN]</code>	Specify one or more search paths where the symbol libraries are located. Multiple library paths must be separated by spaces.
<code>-novasLibs libname1 [libname2 ... libnameN]</code>	Specify one or more symbol library names. Multiple library names should be separated by spaces.

-rule <i>filename</i>	Specify the file name containing the name mapping rules for correlation. This option must be used with the -rtl option.
-target_scope <i>scopeName</i>	Specify the scope to export the mapped file to. This option must be used with the -expMap option.
-top_inport	Include input ports that were successfully mapped from the gate-level design to the RTL design in the top scope. This option must be used with the -expMap option.
-txt <i>filename</i>	When specified, all signals in the specified text file will be mapped from the gate to the RTL for exporting the mapping file. This option must be used with the -expMap option and cannot be used with the -fsdb or the -allreg options.

Examples:

1. Generate a correlation database for the gate design:

```
> crdb -GATE "-f run_gate.f -bas top" -crdb extracted.crdb
```

2. Extract the RTL correlation database and add to the gate extraction, and perform correlation mapping:

```
> crdb -RTL "-f run_rtl.f -bas top" -crdb extracted.crdb \
-rule myrules.lst
```

3. Generate a correlation database for the gate and RTL designs, and perform correlation mapping:

```
> crdb -crdb extracted.crdb -rule myrules.lst \ -GATE "-f run_gate.f
-bas top" -RTL "-f run_rtl.f -bas top"
```

4. Load the specified CRDB and export the mapping according to the export related options.

```
> crdb -crdb crdb.crdb -expMap map.gz -allreg
```

Note:

The mapping file exported with the **-expMap** option is in the gzip format. Decompress the gzip file before reading the mapping file.

5. If RTL KDB contains an upper hierarchy than Gate KDB and if the Gate design top is dut, then you can use the **-top** option as in the following example:

```
crdb -crdb g2r.crdb \
-RTL "-lib rtl_kdb -top dut" \
```

```
-GATE "-lib gate_kdb -top dut" \  
-allreg -target_scope dut.usub -expMap g2r.list.gz
```

If RTL FSDB is generated from a test bench, and you would prefer to let the hierarchy in mapping file be the same as RTL FSDB, then you can use virtual top method as below example:

```
crdb -crdb g2r.crdb \  
-RTL "-lib rtl_kdb -vtop rtl_vtop"\  
-GATE "-lib gate_kdb -vtop gate_vtop" \  
-allreg -target_scope dut.u_sub -expMap g2r.list.gz  
where the content of rtl_vtop and gate_vtop are:  
rtl_vtop :  
dut = tb.udut  
gate_vtop :  
dut = tb.udut
```

Exclude Module for Macro Cell

Some macro cell behavior models contain register description and the register is also mapped when the `-excludeModule` option is not set. This also affects the counting base of CRDB mapping rate. However, the macro cell liberty file (.db) contains the timing and the power information for timing and power analysis tools. Therefore, it needs to only map its boundary ports for macro cell. The macro cell behavior model contains register descriptions. The `-excludeModule` option can exclude the register mapping when performing the CRDB mapping.

Example:

Exclude module list file, for example the (ex_mod.list) as follows:

```
SRAM_128X16  
SRAM_256X8  
PLL_FN_TOP  
%> crdb -RTL "-lib rtl_kdb -vtop rtl_vtop" \  
-GATE "-lib gate_kdb -vtop gate_vtop" \  
-impSVF <SVF files> -excludeModule ex_mod.list \  
-allreg -target_scope dut.u_sub -expMap g2r.list.gz
```

Filtering for Gate Only Register

If the register is added in the gate level and it does not exist in RTL, such as dummy cell or DFT logic, then it impacts the mapping rate.

You can use the `-filterModule` or `-filterInstance` options to filter the flip-flops.

Example:

Filter module list file, for example (flt_mod.list) as follows:

```
sub1_Test_Controller
sub1_SPARE
%> crdb -RTL "-lib rtl_kdb -vtop rtl_vtop"
-GATE "-lib gate_kdb -vtop gate_vtop" \
-impSVF <SVF files> -filterModule flt_mod.list\
-allreg -target_scope dut.u_sub -expMap g2r.list.gz
```

Filter instance list file, for example (flt_inst.list) as follows:

```
dut.u_sub.usub1_Test_Controller
dut.u_sub.usub1_SPARE
%> crdb -RTL "-lib rtl_kdb -vtop rtl_vtop" \
-GATE "-lib gate_kdb -vtop gate_vtop" \
-impSVF <SVF files> -filterInstance flt_inst.list \
-allreg -target_scope dut.u_sub -expMap g2r.list.gz
```

CRDB Environment Variable

CR_SKIP_HIER_FILE

Description

Specify a file that lists a set of hierarchy names to skip in the gate design to correlate to RTL. If the * character is specified at the end of the hierarchy name, all modules with instance names that begin with the part before the * character of the hierarchy name are skipped.

Syntax

```
CR_SKIP_HIER_FILE = filename
```

Example

```
setenv CR_SKIP_HIER_FILE hier_name.txt
```

NOVAS_CR_VHDL_ORIG_CASE

Description

Set the value of this environment variable to 1 to save the name of VHDL design objects to CRDB with the original case in the design. Set the value of this environment variable to 0 to convert the name of VHDL design objects to lower case before saving it to CRDB. The default is 0.

Syntax

```
CR_VHDL_ORIG_CASE = 0|1
```

Example

```
setenv CR_VHDL_ORIG_CASE 1
```

NOVAS_DISPLAY_INVERSE_CORRELATION

Description

Set this environment variable to show inversion with correlation annotation. The default is 0.

Syntax

```
NOVAS_DISPLAY_INVERSE_CORRELATION = 0|1
```

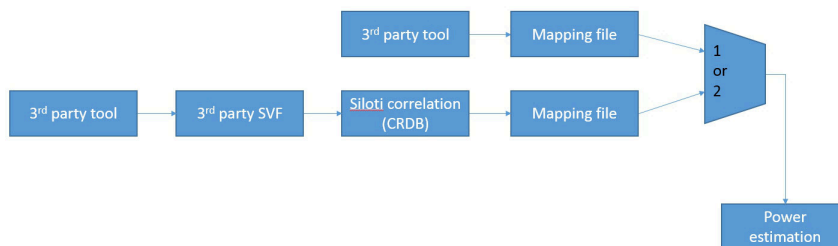
Example

```
setenv NOVAS_DISPLAY_INVERSE_CORRELATION 1
```

Third-Party Tool with Siloti Correlation

If the third-party synthesis and equivalence checker are used for mapping, the equivalence checker mapping file must be converted into the mapping file format that can be used by the Siloti Power solution. The third-party equivalence checker must provide the mapping quality/mapping rate report that describes how many register signals have been mapped. However, note that any step that is performed with the third-party equivalence checker may cause lower mapping rate and less accurate power result.

Figure 7 Third-Party Tool with Siloti Correlation



It is not recommended to use the Siloti Correlation in the third-party synthesis because the result is not guaranteed. Nevertheless, if the flow is adopted, ensure to check or push the equivalence checker to dump SVF file. It is likely that the manual work is required to improve the mapping rate by using mapping rules or other technique. Also, this flow may cause the lower mapping rate and less accurate power result.

Cross-Abstraction Debug

The Siloti Correlation performs seamless debug automation across abstraction levels including value annotations and tracing. A gate-level netlist is typically hard to understand when generated by synthesis tools. To easily understand the meaning of gate-level logic, it is important to have an ability to correlate gate-level verification results back to the RTL design descriptions. The abstraction correlation module can be applied in the background to map gate-level signal data to the RTL design for easier debug and understanding.

The abstraction correlation module inter-operates seamlessly with the data expansion engine to automatically enable analysis and debug of the gate-level results with full visibility on the RTL design.

The combination of the data expansion engine plus the abstraction correlation module working transparently on a limited amount of signal data, enables debug automation and makes it possible to trace problems on the RTL design. The abstraction correlation module maps gate-level signal data captured from the registers to the corresponding RTL registers. The mapped data is then processed with the data expansion engine to make RTL combinational values visible. The combined outcome is RTL-centric debug with high observability of most internal signals of the emulated design. An optional step to improve emulation performance is to use the Visibility Analysis engine to reduce the number of registers dumped during each run. With appropriate emulator interfaces, this step can be user controlled via the emulation setup. For more information, refer to the *Verdi and Siloti Command Reference* and *Siloti User Guide and Tutorial*.

3

Siloti Correlation Practice

This chapter is intended to quickly introduce you to applications where you can use the Siloti correlation features holistically and to familiarize you with some core concepts and use models.

- [Before You Begin](#)
- [Debug Gate-Level Results on the RTL Design](#)
- [Verdi Power Analysis Acceleration Flow](#)

Before You Begin

Before you begin the tutorial, you (or your system administrator) must install the Verdi and Siloti systems as described in the accompanying *Installation and System Administration Guide*.

Note:

The optional `Novas-2016??-demo.tar.gz` package must be installed (where 2016 corresponds to the year and ?? corresponds to the month).

You must also complete the following actions in order to set up the Siloti environment and the files required for tutorials:

1. Add the Verdi application (binary) to the search path and specify the search path to the license file.

Refer to the *Setting Up the Environment and Running the Software* section in the *Installation and System Administration Guide* for details.

2. Create a working directory:

```
% mkdir <working_dir>
```

3. All of the tutorial data resides in the `$NOVAS_HOME/demo` directory. Make a copy of these demo files in your working directory:

```
% cp -r <NOVAS_INST_DIR>/demo <working_dir>
```

Debug Gate-Level Results on the RTL Design

Gate/RTL correlation is relevant to any of the applications described previously. For example, the emulation system is frequently only aware of the gate-level netlist and thus, produces results matching the netlist. This application allows the results to be mapped back to the RTL design and expanded to enable debug on the original RTL design.

This chapter consists of the following sections:

- [Input Requirements](#)
- [Extract Correlation Database](#)
- [Create Mapping File \(Optional\)](#)
- [Expand Data and Debug on the RTL Design](#)

Input Requirements

For RTL/gate correlation, the following items are required:

- RTL design
- Gate-level design with symbol libraries
- Partial or full gate-level simulation results file in either FSDB or VCD format

Extract Correlation Database

You can extract CRDB using the command line:

1. Change to the gate directory as follows:

```
% cd <working_dir>/demo/siloti/gate
```

2. Set the environment variables for the symbol library using the following commands:

```
setenv NOVAS_LIBS slow
setenv NOVAS_LIBPATHS
```

3. Set the environment variables for the symbol library using the following commands:

```
% crdb -RTL "-f run.f -path ../rtl/" -GATE "-f run.f" -crdb
mapped.crdb
```

Create Mapping File (Optional)

You can create a mapping file to map the gate-level FSDB data for essential signals to the RTL design. The correlation engine performs this automatically by analyzing the gate-level design, the RTL design, and the results in the gate-level FSDB file.

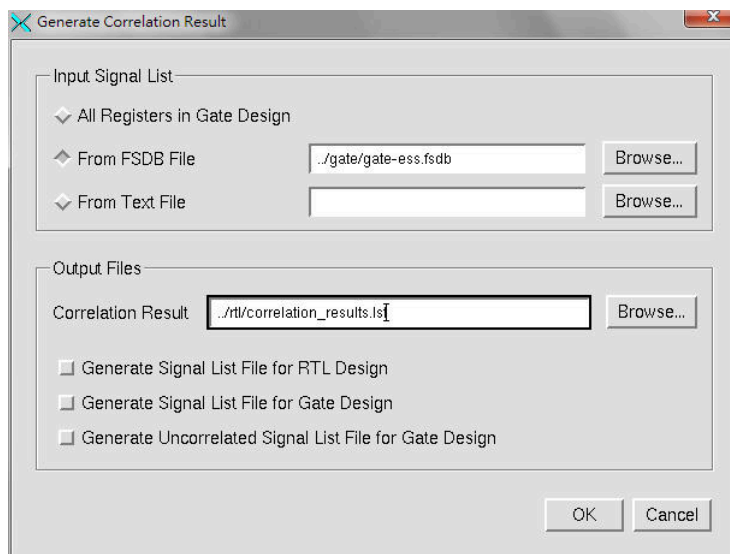
1. Execute the following command to bring invoke Verdi

```
% verdi &
```

2. Invoke the command **Tools -> Visibility -> Gate/RTL Correlation -> Advanced Commands -> Load CRDB File** from the main window to load extracted CRDB.
3. In the same session, select the **Tools -> Visibility -> Gate/RTL Correlation -> Generate Result** command to open the *Generate Correlation Result* form.
4. In the **Input Signal List** section, select the **From FSDB File** option and enter `../gate/gate-ess.fsdb` in the text field. You can also click the related **Browse** button to open the *File Manager for the FSDB* form and view the directory structure to locate the FSDB file.
5. In the **Output Files** section, specify `../rtl/correlation_results.lst` in the **Correlation Result** text field. You can also click the **Browse** button to open the *File Manager for Output* form and view the directory structure to specify the location and file name.

The following figure shows the *Generate Correlation Result* form:

Figure 8 The Generate Correlation Result Form



6. Click the **OK** button to create the mapping file, `correlation_results.lst`.

Note:

For details on creating and using additional mapping rules, see the Appendix A: [Name Mapping Rules](#) chapter.

Now that the mapping file is created, you can use it with any FSDB file generated from the same gate-level design.

7. Choose the **File -> Exit** command in the main window to exit.

Expand Data and Debug on the RTL Design

Now that a mapping file is created, the results data is correlated and expanded on-the-fly relative to the original RTL design description using the following steps:

1. Change to the RTL directory using the following command:

```
% cd <working_dir>/demo/siloti/rtl
```

2. Execute the following command to load the design and the essential signal dump FSDB file:

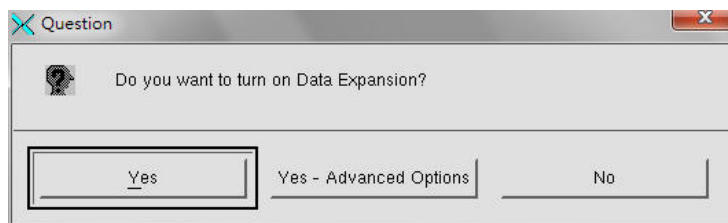
```
% verdi -f run.f -ssf ../gate/gate-ess.fsdb -ba -ba_mode mbba &
```

Note:

1) The `-ba` switch uses the Behavior Analysis settings from the `novas.rc` resource file unless you specifically include them on the command line. For a list of Behavior Analysis options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*. 2) The `-ba_mode` switch specifies the default mode for Behavior Analysis. The suggested mode for Data Expansion is Module Based Behavior Analysis (mbba). The mode is also specified by the environment variable, see the *Behavior Analysis* section of *Appendix A* in the *Verdi and Siloti Command Reference* for details.

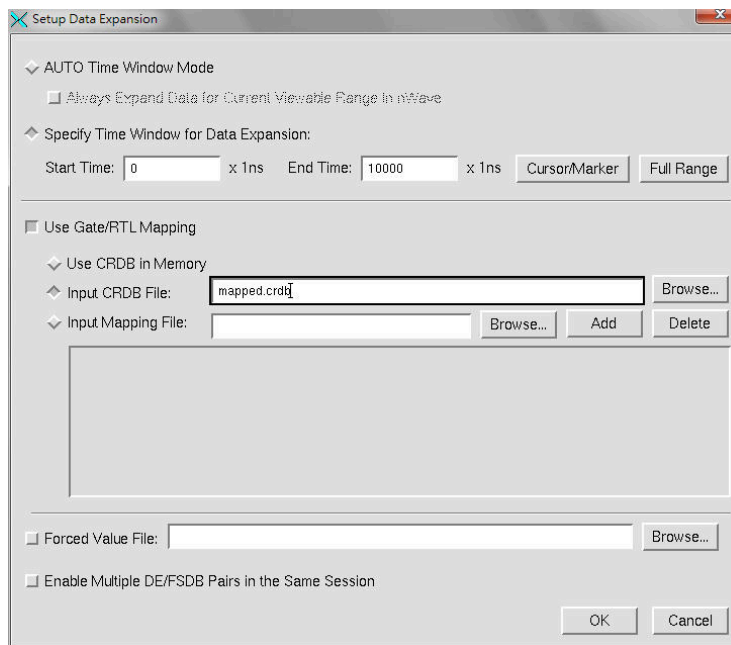
3. In the Data Expansion *Question* dialog, click **Yes-Advanced Options** button. This opens the *Setup Data Expansion* form.

Figure 9 *The Data Expansion Question Dialog*



4. In the *Setup Data Expansion* form, enable the **Specify Time Window for Data Expansion** option and enter 0 in the **Start Time** text field and 10000 (ns) in the **End Time** text field.
5. Turn on the **Use Gate/RTL Mapping** option.
6. Select the **Input CRDB File** option and enter `mapped.crd` in the associated field. The following figure shows the *Setup Data Expansion* form:

Figure 10 The Setup Data Expansion Form With Mapping Enabled.



Alternatively, you can enable the **Input Mapping File** option and enter `correlation_results.lst` in the associated text field and then click the **Add** button.

7. Click the **OK** button.

With the RTL design and expanded simulation results, you can use all of the advanced debug capability available in the Verdi platform. For examples of using the expanded data, see the *Debug With Data Expansion* section in one of the previous applications.

Note:

The above GUI steps is the same as adding **-DE "-crdbfile mapped.crd -start_time 0 -end_time 10000"** or **-DE "-g2r_mapping correlation_results.lst -start_time 0 -end_time 10000"** on the command line to setup Data Expansion with the specified options. For a list of `-DE` options, see the *verdi* utility description in the *Verdi and Siloti Command Reference*.

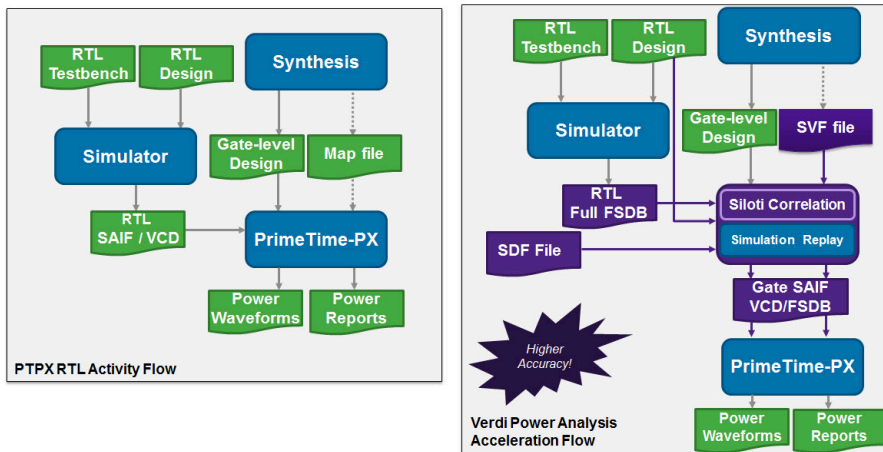
The Gate-to-RTL Correlation and Data Expansion engines enable the debug of gate-level results in the context of the RTL design. They provide an efficient means of locating and isolating problems detected during silicon validation, emulation, or full-chip gate-level simulation.

Verdi Power Analysis Acceleration Flow

Verdi power analysis acceleration flow runs the simulation in earlier stage similar to RTL activity analysis in the flow but with faster run times. It also delivers highly accurate power metrics with gate-level analysis accuracy.

In the past, adopting the power analysis in SoC verification flow, you need to run the gate-level simulation to obtain the accurate power analysis. Thus, the accuracy is achieved late in the design flow with long runtime. This flow performs the following steps:

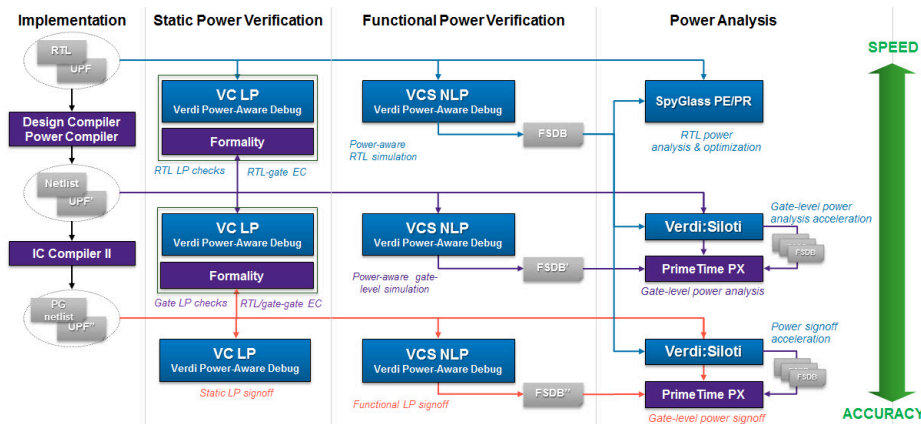
1. **Average power analysis:** the gate-level netlist and switching activity, such as, VCD and SAIF for the vector free power analysis
2. **Cycle Accurate Peak Power:** the gate-level netlist and RTL VCD for the RTL activity analysis
3. **Peak Power at VCD Resolution:** the gate-level netlist and gate-level VCD for the gate-level analysis



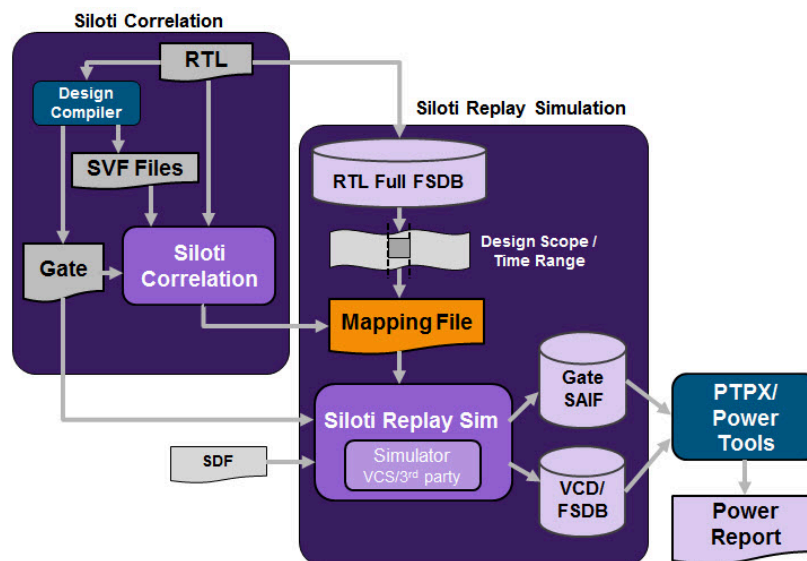
RTL Activity of PTPX Flow and Verdi Power Analysis Acceleration Flow

Verdi power analysis acceleration flow leverages Siloti correlation and Siloti replay simulation technologies, using the available RTL and synthesis information to accelerate power analysis. The Siloti correlation uses RTL and netlist designs, leverages SVF output from the design compiler, and generates the mapping file. The Siloti Replay Simulation

uses the RTL simulation dump, restricts the simulation and dumping, leverages the SDF file, and re-generates the gate-level data.



Siloti Correlation and Replay Simulation in Power Analysis Flow



Verdi Power Analysis Acceleration Flow Leverages Siloti Correlation and Replay Technologies

With Siloti correlation and Siloti replay simulation technologies, the following advantages are available to help the Verdi power analysis acceleration flow:

- For higher accuracy:
Runs live gate-level simulations.
Uses the POR simulator from the user.

Chapter 3: Siloti Correlation Practice

Verdi Power Analysis Acceleration Flow

Incorporates the SDF information.

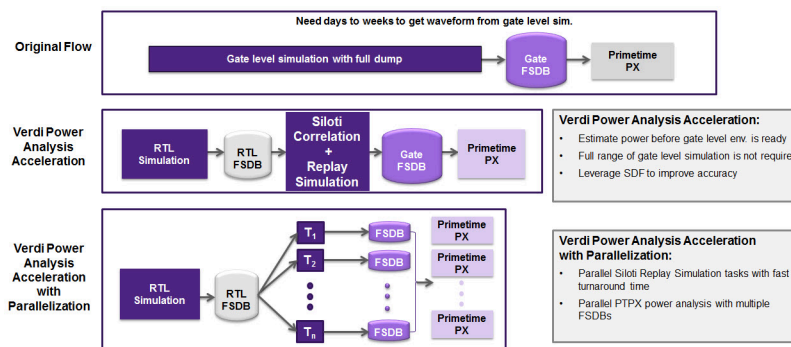
Leverages the SVF mapping file with higher quality

- Faster runtimes:

Restrict the simulation to desired time periods and design scopes.

Applies parallelization technology.

Verdi power analysis acceleration flow has these advantages because it estimates the power before the gate-level environment is ready. This way, the full range of gate-level simulation is not required. Also, this flow leverages the SDF to improve the accuracy for power analysis. In addition, further parallelization helps speed up the flow.



Comparison of SoC Verification and Verdi Power Analysis Acceleration Flows

4

Name Mapping Rules

This chapter describes the syntax for name mapping rules. The user can access the *Perform CRDB Correlation* form by invoking **Correlation -> Gate/RTL Correlation -> Advanced Commands -> Perform CRDB Correlation** from the main Verdi and Siloti process (with the RTL design loaded). Refer to the *Perform CRDB Correlation* description in the *Visibility* chapter of the *Verdi and Siloti Command Reference* for complete details.

Figure 11 *Perform CRDB Correlation Form - Reg Exp Tab*

The screenshot shows the 'Prepare CRDB' dialog box with the 'Reg Exp Tab' selected. The dialog has four tabs: 'Default Rules', 'Hierarchy Change Rules', 'Module Specific Rules', and 'Scan Cell Rules'. The 'Regular Expressions' section contains two text input fields: 'Gate Matching Exp:' and 'RTL Substitution String:'. Below these fields are three buttons: 'Add', 'Insert', and 'Replace'. A large empty list box is present, with 'Up', 'Down', 'Delete', and 'Delete All' buttons to its right. The 'Pre-defined Rules' section contains three text input fields: 'Hierarchical Separator:' (containing '/'), 'Tri-state:' (containing '%s_tri'), and 'Register:' (containing a complex regex pattern). At the bottom are 'Load...', 'Save...', 'Finish', and 'Exit' buttons.

Select the **Default Rules** tab to specify name mapping rules for your design.

In this chapter:

- [Regular Expression Examples](#)
- [Pre-defined Rules Examples](#)
- [Applying Rules](#)

Regular Expression Examples

Using the **Regular Expression** section on the **Default Rules** tab in the *Perform CRDB Correlation* form, the following rules will result in the described substitutions.

Figure 12 Regular Expression Examples

Regular Expressions

Gate Matching Exp:

RTL Substitution String:

u%d => m@1
abc_([A-Z]+)_%d => cba_@2_@1
^top => tb.newTop
pre_ =>
u1 => m2

1. Matching string is “u1” and substitution string is “m2”

Gate Matching Exp field contains: u1

RTL Substitution String field contains: m2

Causes strings like “top.u1.s1” to become “top.m2.s1”.

2. Match on the first %d and replace with @1

Rule: u%d => m@1

Causes “u1” to be mapped to “m1” and “u5” to be mapped to “m5”.

3. Match on string combination, then replace and change order.

Rule: abc_([A-Z]+)_%d => cba_@2_@1

Causes strings like “abc_PQR_555” to be mapped to “cba_555_PQR”.

4. If your gate-level design top does not match your RTL, you can use this rule to create a 'virtual top'.

Rule: ^top => tb.newTop

Causes strings like "top.u1.s1" to be mapped to "tb.newTop.u1.s1".

5. Delete the string by leaving the substitution string 'empty'.

Rule: pre_ =>

Causes strings like "top.pre_u1.s1" to be mapped to "top.u1.s1".

Note:

Regular Expressions work on a single design name only; they do not work for names crossing multiple hierarchies with delimiters. For example, assuming a full name of u1.u2.\escaped_name.abc.dfg.q, the rule u2.%s.q => u2.q will not work since the left hand side crosses multiple hierarchies but the rule abc.dfg => xyz will work since the left hand side belongs to a single name.

Pre-defined Rules Examples

The **Pre-defined Rules** on the **Default Rules** tab of the *Perform CRDB Correlation* form follows the regular expression name matching and substitution string syntax; however, the data types are specially treated. The pre-defined rules support three gate-level to RTL correlation types: hierarchical separator, tri-state devices and registers.

Figure 13 *Perform CRDB Correlation Form - Pre-defined Rules*

1. **Hierarchical Separator** is the separator for flattened signal/instance. Only one character is allowed.

Example 1:

RTL uses: top.u1.u2.sig[0]

Gate uses: top.\u1!u2!sig[0]

Specify the hierarchical separator as '!'.

Example 2:

RTL uses: top.u1.u2.s1

Gate uses: top.u1#u2#s1

Specify the hierarchical separator as '#'.

Note:

Common hierarchy separators such as underscore '_' and dot '.' do not need to be specified as they are handled internally.

2. **Tri-state** is the format for tri-state instance and tri-state output net.

Example:

Gate uses: bus1_tri

Specify the rule as: %s_tri

Causes the gate "bus1_tri" signal to be mapped to "bus1" in the RTL.

3. **Register** is the format for register instance and register output net only. The register substitution consists of the register string and the register index format. Example:

Gate uses: ccu_fetch_reg_3_

Specify the rule as: %s_reg_%d_

Causes the gate "ccu_fetch_reg_3_" signal to be mapped to signal "ccu_fetch[3]" in the RTL.

Note:

A rule in the **Register** or **Tri-state** field can only contain one '%s' and it must be at the beginning of the rule. For example, the rule "%s_reg_%d" is valid but the rule "%s_reg_%d" is not.

Applying Rules

The rules will be applied sequentially in the order listed in the mapping rule table (both regular expression and pre-defined rules will be applied).

Example 1:

Assuming the **Regular Expression** rules are:

a => x

b => y

`c => z`

and the **Pre-defined** rules are:

Hierarchical Separator: `#`

Register: `%s_reg_%d_`

This causes the register output net “top.m_a.m_b#m_c#test_reg_3_” to be mapped to “top.m_x.m_y.m_z.test[3]”.

Example 2:

Assuming the **Regular Expression** rules are:

`a => x`

`ab => xy`

This causes the second rule to become useless because the `a=>x` rule will be applied first (all ‘a’ will be replaced by ‘x’, so there will not be any ‘ab’ strings).

Example 3:

Assuming the **Register** field of the **Pre-defined** rules include:

`%s_Z_%d`

and a library cell for a register is defined as:

```
FF abc_Z_6(.CK(clk), .Q(n123), .QN(n124), .RN(reset));
```

This causes the library cell `abc_Z_6` to be mapped to the RTL register signal `abc[6]`. If either the simulation library or the symbol library is available for the flip-flop (FF), the register output nets, `n123` and `n124`, will also be mapped to `abc[6]` and its inversion respectively. In addition, the `clk` and `reset` signals will be mapped according to the attributes of the CK and RN pins they are connected to and the syntax of the RTL code that defines register `abc[6]`.

Note:

It is very common for an RTL register net to be synthesized to a gate-level flip-flop or latch library cell; therefore, the recommendation is to first correlate a register library cell to its corresponding register net using the **Regular Expression** and/or **Pre-defined** rules. Then the correlation algorithm will correlate the nets connected to the cell according to the pin attributes of the cell from either the symbol library or the simulation library.